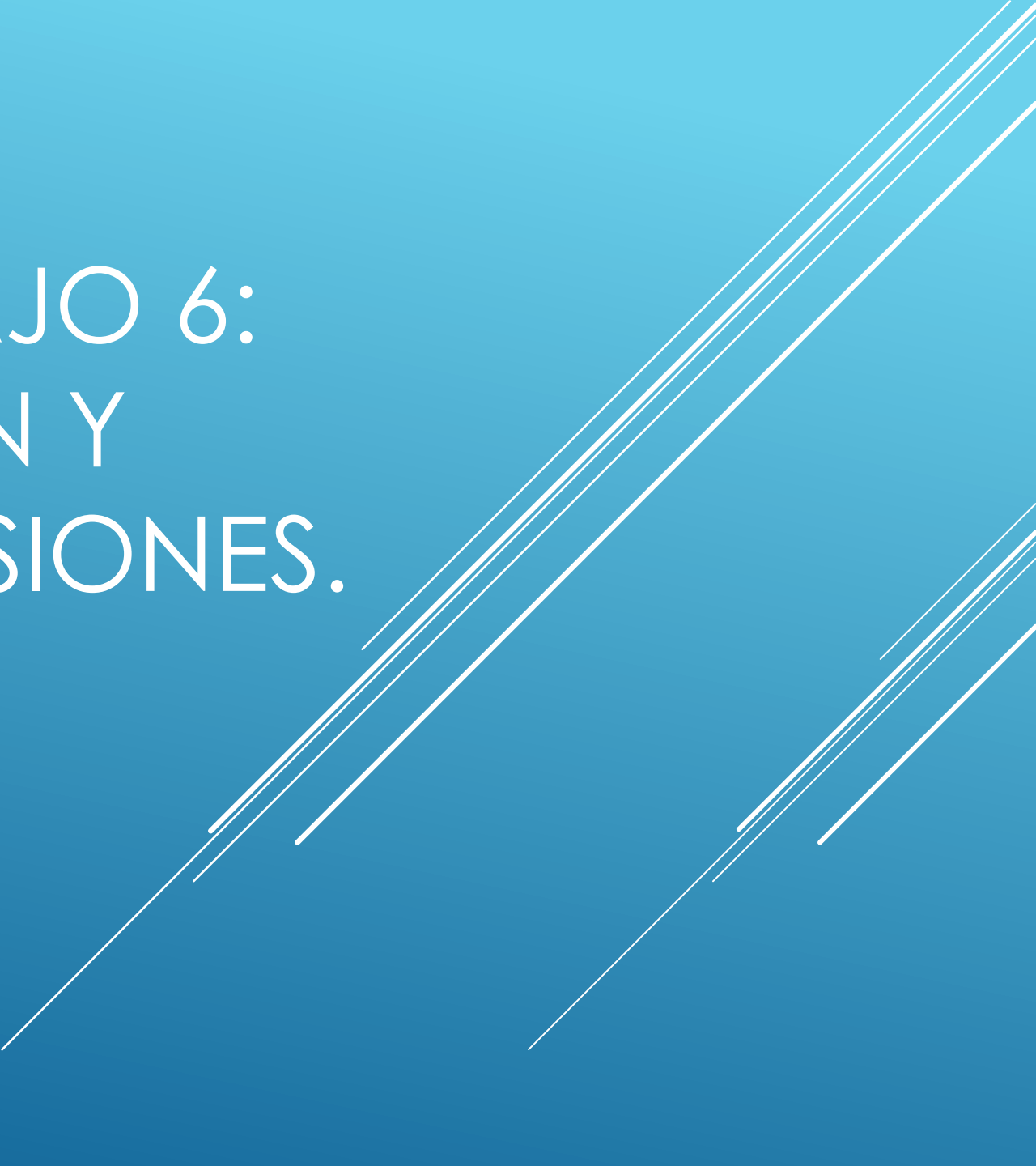


UNIDAD DE TRABAJO 6: DOCUMENTACIÓN Y CONTROL DE VERSIONES.



DOCUMENTACIÓN DE APLICACIONES WEB

- La **documentación** de un proyecto de software es tan **importante** como su **código**.
- Conviene **documentar** en una aplicación **tres** aspectos fundamentales de la aplicación:
 1. **La interfaz:** **Qué hace** (no como lo hace) una **función** o un **método** de una **clase**, qué parámetros hay que pasar y qué devuelve. Esta información es tremendamente útil para las personas que **utilizan** funciones o clases diseñadas por otros.
 2. **La implementación:** Indicar **cómo** está implementada cada **función**, cómo se lleva a cabo cada paso, por qué se utiliza determinada variable, qué **algoritmo** se **utiliza**, qué hacen los métodos privados de una clase. Toda esta información resulta interesante a quienes tengan que **depurar** o **actualizar** bloques de código de la aplicación.
 3. **La toma de decisiones:** **Por qué** se ha **implementado** de **determinada forma** y no de otra la aplicación, por ejemplo, para analizar el **rendimiento** de la aplicación y optimización de recursos. Esto resulta **interesante** a **nivel de implementación** para los **desarrolladores** y a **nivel funcional** para los **responsables del desarrollo**.

Existen algunas herramientas que permiten generar documentación de forma automática a partir del código fuente. **Javadoc** es la herramienta estándar en **Java**. Para **PHP** una de las herramientas más utilizadas es **phpDocumentor**.

PHPDOCUMENTOR

- Una **buena** documentación nos facilita, el **mantenimiento** futuro de la aplicación. Es útil **saber** lo que hacen las partes que **desarrollan otras personas**.
- Mediante **comentarios** y unas **etiquetas** especiales **definimos** de forma sencilla qué hace cada clase, cada método y cada función de nuestro código.
- Genera la documentación desde **línea de comandos**.
- Permite generar la documentación en varios **formatos**: HTML, PDF y XML.
- Una alternativa a la colección de código PHP **phpDocumentor** es el programa **Doxygen**.



FUNCIONAMIENTO DE PHPDOCUMENTOR

- En phpDocumentor la documentación se distribuye en bloques "**DocBlock**". Estos bloques siempre se colocan justo antes del elemento al que documentan y su formato es:

```
/**
 * Descripción breve (una línea)
 *
 * Descripción extensa. Todas las líneas que
 * sean necesarias
 * Todas las líneas comienzan con *
<- Esta línea es ignorada
 *
 * Este DocBlock documenta la función suma()
 */
function suma()
{
    ...
}
```

- Dentro de cada **DocBlock** se pueden incluir **marcas** que serán interpretadas con un **significado especial**: @access, @author, @copyright, @deprecated, @example, @ignore, @internal, @link, @see, @since y @version.
- Otras marcas** que solamente se pueden utilizar en bloques de **determinados elementos**: @global, @param y @return.



PHPDOCUMENTOR

Resumiendo, las **etiquetas** más usuales:

- **@autor** autor del código.
- **@copyright** información sobre derechos.
- **@deprecated** para indicar que el elemento no debería utilizarse, ya que en futuras versiones podría no estar disponible.
- **@example** permite especificar la ruta hasta un fichero con código PHP.
- **@ignore** evita que phpDocumentor documente un determinado elemento.
- **@link** para incluir un enlace (<http://...>) a un determinado recurso.
- **@see** se utiliza para crear enlaces internos (enlaces a la documentación de un elemento).
- **@since** permite indicar que el elemento está disponible desde una determinada versión del paquete o distribución.
- **@versión** versión actual del elemento.
- **@global** para especificar el uso de variables globales dentro de una función.
- **@param** para documentar parámetros que recibe una función.
- **@return** valor devuelto por una función.



INSTALACIÓN DE PHPDOCUMENTOR

- Como **requisito** previo **Php** y **Apache** deben de estar funcionando correctamente.
- Para la **instalación** ejecutar como root los siguientes comandos:
 1. `wget https://phpdoc.org/phpDocumentor.phar`
 2. `chmod +x phpDocumentor.phar`
 3. `mv phpDocumentor.phar /usr/local/bin/phpDocumentor`
 4. `mkdir /var/www/html/docs`
- Tenemos phpDocumentor listo para **funcionar**.



CONFIGURACIÓN DE PHPDOCUMENTOR

- Una vez hemos instalado phpDocumentor se puede trabajar con el para generar automáticamente la documentación de nuestros proyectos PHP, se trabaja desde línea de comandos.
- **Ejecutar:**

```
phpDocumentor run -d [carpeta_donde_estan_los_proyectos_php] -t [carpeta_donde_se_almacena_los_archivos_de_documentación]
```

Ejemplo:

```
phpDocumentor run -d /var/www/html/PHP/ -t /var/www/html/docs/
```

Con el navegador abrir **index.html** de la documentación.

Ejemplo: <http://localhost/docs/index.html>



JAVADOC

- **Documentar** con suficiente detalle un proyecto es muy **importante** para realizar mantenimientos futuros, conocer qué hacen las clases, métodos, interfaces, variables, etc. y cómo lo hacen.
- **Javadoc** es una utilidad de **Sun Microsystems** para generar **APIs** (Application programming Interface) en formato **HTML** de un archivo de código fuente Java.
- La herramienta Javadoc **extrae** los **comentarios** del código fuente de los programas Java y genera con ellos un juego de documentación en formato **html**.
- La documentación a ser utilizada por Javadoc se escribe en comentarios que comienzan con **/**** y que terminan con ***/**
- Las **etiquetas** se ubican dentro de los comentarios, comienzan con el símbolo **@** y son sensibles a mayúsculas-minúsculas.
- Los comentarios JavaDoc están destinados a **describir**, principalmente, **clases** y **métodos**.
- Es el estándar para documentar, de forma automática, clases de Java, por lo que se encuentra en la mayoría de los **IDE** específicos para Java (**Eclipse, NetBeans...**)



JAVADOC

Resumiendo, las **etiquetas** más usuales:

- **@author** nombreDelAutor descripción. Indica quién escribió el código al que se refiere el comentario.
- **@version** númeroVersión descripción. Si se quiere indicar la versión. Normalmente se usa para clases, pero en ocasiones también para métodos.
- **@param** nombreParámetro descripción. Para describir un parámetro de un método.
- **@return** descripción. Describe el valor de salida de un método.
- **@see** nombre descripción. Cuando el trozo de código comentado se encuentra relacionada con otra clase o método, cuyo nombre se indica en nombre.
- **@throws** nombreClaseExcepción descripción. Cuando un método puede lanzar una excepción ("romperse" si se da alguna circunstancia) se indica así.
- **@deprecated** descripción. Indica que el método (es más raro encontrarlos para una clase) ya no se usa y se ha sustituido por otro.



SISTEMAS DE CONTROL DE VERSIONES

- Con un sistema de control de versiones hay un **directorio**, controlado por esta herramienta, donde se van guardando los **fuentes** de nuestro proyecto con todas sus versiones.
- Utilizan una **arquitectura cliente-servidor** en donde el servidor guarda la(s) versión(es) actual(es) del proyecto y su historia.
- El **funcionamiento general** de un sistema de control de versiones sigue el siguiente ciclo de operaciones:
- **Descarga de ficheros inicial (Checkout):**
 - El primer paso es bajarse los ficheros del repositorio.
 - El "checkout" sólo se hace la primera vez que se usan esos ficheros.
- **Ciclo de trabajo habitual:**
 - Modificación de los ficheros.
 - Actualización de ficheros en local (Update).
 - Resolución de conflictos (si los hay).
 - Actualización de ficheros en repositorio (Commit).



SISTEMAS DE CONTROL DE VERSIONES

Entre las funciones que permite realizar un sistema de control de versiones cabe destacar:

- Sacar **copias** del proyecto al mismo tiempo.
- Realizar cambios a los ficheros manteniendo un **histórico** de los **cambios**.
- Los clientes pueden también **comparar** diferentes **versiones** de archivos.
- **Unir cambios** realizados por diferentes **usuarios** sobre los mismos ficheros.
- Sacar una "**foto**" histórica del proyecto tal como se encontraba en un **momento** determinado.
- **Actualizar** una copia **local** con la última versión que se encuentra en el **servidor**.
- Mantener distintas **ramas** de un proyecto.



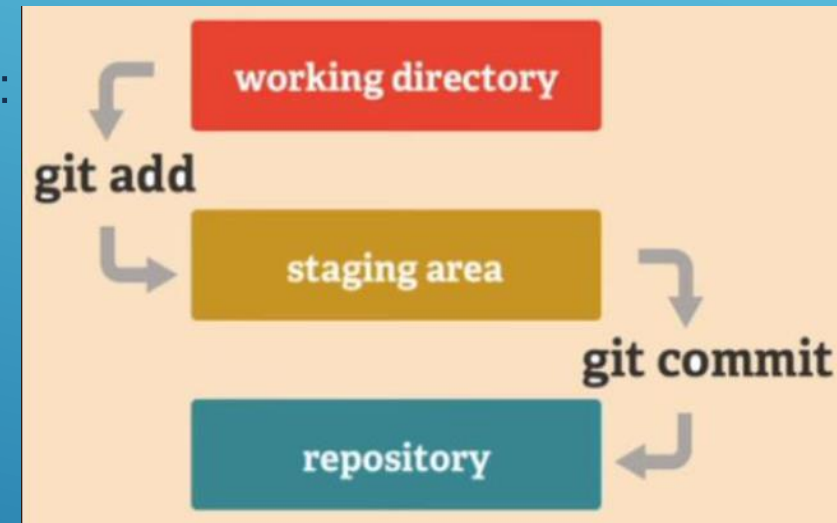
GIT COMO SISTEMA DE CONTROL DE VERSIONES

- **Git** creado en **2005**, es tremendamente **rápido**, muy **eficiente** con **grandes** proyectos, y tiene un increíble sistema de **ramificación** (branching) para desarrollo no lineal.
- Cada vez que confirmas un **cambio**, o **guardas** el **estado** de tu proyecto en Git, él básicamente hace una "**foto**" del aspecto de todos tus archivos en ese momento, y **guarda** una **referencia** a esa instantánea. Para ser eficiente, si los archivos no se han modificado, Git no almacena el archivo de nuevo, sólo un **enlace** al archivo anterior idéntico que ya tiene almacenado.
- La **mayoría** de las **operaciones** en Git sólo necesitan archivos y recursos **locales** para operar.
- **GitHub** es un servicio de **hospedaje web** para proyectos que utilizan el sistema de control de versiones Git. GitHub ofrece tanto planes **comerciales** como planes **gratuitos** para proyectos de código **abierto**.



FUNCIONAMIENTO DE GIT

- Git tiene **tres estados** principales en los que se pueden encontrar los archivos:
 - **Modificado** estado en el que se ha modificado el archivo pero todavía no se ha confirmado a la base de datos.
 - **Preparado** significa que se ha marcado un archivo modificado en su versión actual para que vaya en la próxima confirmación.
 - **Confirmado** significa que los datos están almacenados de manera segura en la base de datos local.
- Esto nos lleva a las **tres secciones** principales de un proyecto de Git:
 - El directorio de **trabajo** (working directory)
 - El área de **preparación** (staging area)
 - El directorio de **Git** (Git directory)
- El **flujo de trabajo** básico en Git consiste en:
 - **Modificar** una serie de archivos en el directorio de trabajo.
 - **Preparar** los archivos, añadiendo instantáneas de ellos al área de preparación.
 - **Confirmar** los cambios, lo que toma los archivos tal y como están en el área de preparación, y almacena esa **instantánea** de manera permanente en el directorio de Git.



FUNCIONAMIENTO DE GIT

